B.SC. COMPUTER SCIENCE CAPSTONE PROJECT • 2026

StudyMate: AI-Powered Study Assistant

Complete Viva Voce Defense Guide & Architectural Dossier

A comprehensive, exhaustive preparation manual for external examiners, academic evaluators, and final project defense. Contains complete system architecture, data models, algorithm workflows, security mechanisms, code implementations, and 36 critical viva questions with expert model answers.

PRIMARY TECHNOLOGY STACK
Next.js 14, TypeScript, Tailwind CSS

DATABASE & DRIVER
MongoDB v7.6.0 (Native Driver)

INFERENCE ENGINE
Groq Cloud API (LLaMA 3.1)

AUTHENTICATION & SECURITY
NextAuth v4, bcryptjs, Gmail SMTP OTP

Candidate: B.Sc. Computer Science Candidate
Academic Year: 2025–2026
Status: Defense Ready

1. Executive Summary & Project Identification

StudyMate is an intelligent, full-stack academic assistance web application developed as a final year capstone project. It bridges the critical pedagogical gap between passive reading of voluminous study materials (lecture slides, textbook chapters, academic papers) and high-retention active recall.

1.1 Problem Statement

- **Cognitive Overload:** Higher education students are inundated with 50-100 page PDF lecture decks and syllabi, making exam revision slow and inefficient.
- **Passive Learning Trap:** Re-reading notes yields low cognitive retention compared to active recall (flashcards) and self-assessment (practice tests).
- **Generic AI Limitations:** General-purpose AI chatbots (ChatGPT) suffer from hallucinations, lack context grounding, have no structured study tools (auto-grading tests), and present data privacy risks.

1.2 Core Objectives

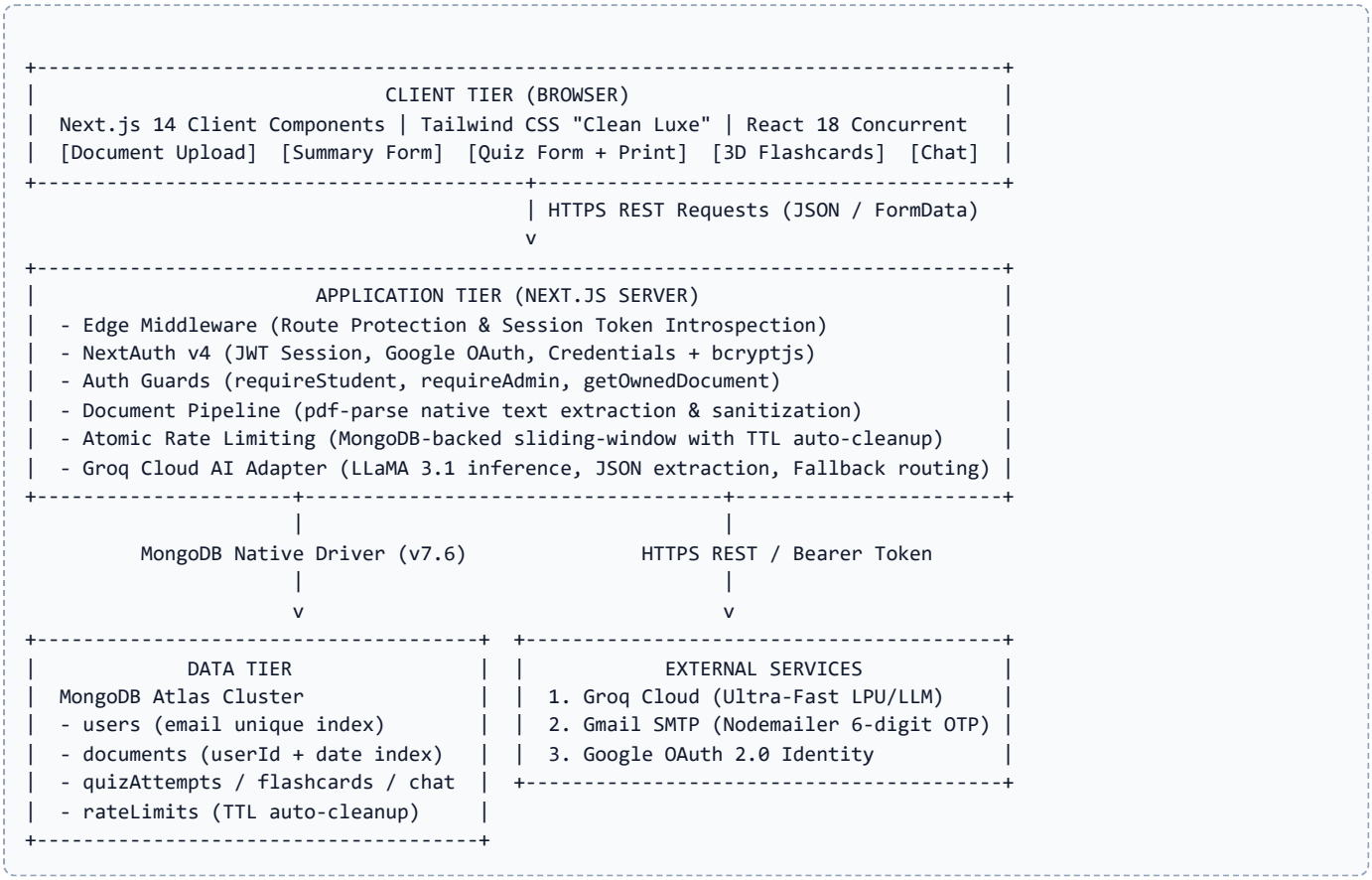
1. Provide authenticated, encrypted PDF document ingestion up to 5MB.
2. Extract native text without reliance on third-party OCR cloud services.
3. Generate chapter-wise, structured AI Summaries with formatted/raw views.
4. Dynamically construct multiple-choice practice quizzes with scoring, answer explanations, and printable scorecards (PDF export).
5. Generate 3D interactive flashcards for active terminology recall.
6. Provide a Retrieval-Augmented Generation (RAG) Document Study Chat that strictly grounds answers in student notes with zero external hallucinations.
7. Empower academic evaluators with an Admin Oversight Dashboard tracking system-wide users, notes, and activity.

> **Key Viva Takeaway:** If an examiner asks *"What makes your app different from standard ChatGPT?"* answer:
> **"StudyMate grounds all generation strictly inside the student's authenticated PDF text using RAG, prevents cross-document data leakage, and provides structured pedagogical modules (timed quizzes, flashcards, printable score reports) that raw chat tools cannot deliver."**

---

2. System Architecture & Tech Stack Justifications

StudyMate follows a modern **Three-Tier Serverless / Edge-Ready Architecture** built on the Next.js 14 App Router paradigm.



2.1 Why Next.js 14 App Router?

Technology	Traditional Approach	Why StudyMate Chose It
Next.js 14 App Router	Separate Express.js backend + CRA React frontend	Single unified TypeScript codebase. Built-in Route Handlers eliminate CORS errors, provide SSR/Server Components for fast LCP, and deploy effortlessly to Vercel.
TypeScript 5	Plain JavaScript	End-to-end static typing. Eliminates runtime bugs in API payloads, database models, and NextAuth session tokens.
MongoDB Native Driver	Mongoose ODM	Mongoose adds significant overhead, schema validation bloat, and connection pooling issues in serverless runtimes. The native driver (v7.6) provides raw performance, atomic pipelines, and zero abstraction lag.
Groq Cloud (LLaMA 3.1)	OpenAI GPT-4 / Local Ollama	Groq's Tensor Streaming Processing (LPU) provides sub-second token generation (300-500 tokens/sec), crucial for interactive quizzes and instant flashcards without student waiting fatigue.
bcryptjs (Pure JS)	Native C++ bcrypt	Native `bcrypt` compiles platform-specific binary bindings (`node-gyp`). In serverless (Linux x64 musl/glibc), binary ABI mismatches trigger fatal HTTP 500 crashes. Pure JS `bcryptjs` is 100% portable.
pdf-parse	Cloud OCR APIs / python-pdf	Fast in-memory node-native PDF stream parsing without spinning up external microservices or incurring third-party API bills.

---

3. Module-by-Module Technical Walkthrough

3.1 Authentication & Security Architecture (Defense-in-Depth)

Authentication is implemented via **NextAuth.js v4** supporting two authentication pathways:

1. **Google OAuth 2.0:** Single-click authentication using verified Google student accounts.
2. **Credentials Provider:** Email and password registration with mandatory **6-digit OTP Email Verification** via Nodemailer and Gmail SMTP.

> **Brute-Force Attack Prevention:** In `lib/auth.ts`, failed password attempts are tracked in the user document. If `failedLoginAttempts >= 6`, the account is automatically locked for 60 seconds (`lockedUntil`), preventing automated dictionary attacks.

> **Dynamic Role Synchronization:** Rather than trusting stale role claims in JWT cookies, the NextAuth `jwt` callback executes a live MongoDB query on token refresh to fetch the latest role, guaranteeing instant revocation if an admin demotes an account.

3.2 Document Ingestion Pipeline (`/api/documents`)

1. The student selects an academic PDF file up to 5MB.
2. The frontend sends a `multipart/form-data` POST request to `/api/documents`.
3. The route handler executes `requireStudent()` auth guard.
4. File MIME type (`application/pdf`) and byte size (`MAX_PDF_SIZE = 5MB`) are strictly validated.
5. `lib/pdf.ts` consumes the file buffer using `pdfParse`, strips irregular spacing, and extracts clean plain text.
6. If no readable text is found (e.g., scanned image PDF), a user-friendly error is returned.
7. The document metadata and extracted text are stored in the MongoDB `documents` collection with the authenticated `userId`.

3.3 AI Summary Generator (`/api/summary`)

- Allows students to choose summary length (Short, Medium, Comprehensive).
- Retrieves the student's owned document text via `getOwnedDocument(userId, docId)`.
- Shortens notes to `MAX_TEXT_CHARS (12,000 chars)` to stay well within model context limits.
- Issues a prompt to Groq demanding structured markdown with:
  - *Executive Overview*
  - *Core Theoretical Concepts*
  - *Key Formulas / Definitions*
  - *Exam Revision Takeaways*
- The UI provides an interactive toggle between a rich formatted view (with callouts and bullet points) and a clean raw markdown view.

3.4 AI Quiz Generator & Printable Scorecard (`/api/quiz`)

- Generates 5 to 30 multiple-choice questions dynamically tailored to the document.
- Uses Groq with `jsonMode: true` (enforcing `response_format: { type: "json_object" }`).
- Each question contains: `question`, `options` (array of 4), `answer` (0-3 index), and an academic `explanation`.
- Students take the interactive quiz with real-time progress indicators.
- Upon completion, the attempt score is saved in MongoDB (`quizAttempts`).
- **Print / PDF Export:** A dedicated "Download Quiz Result (PDF)" button triggers `window.print()` with print-specific CSS rules (`@media print`) that hide navigation sidebars, format question reviews cleanly, and prevent awkward page breaks (`page-break-inside: avoid`).

3.5 3D Spaced-Repetition Flashcards (`/api/flashcards`)

- Extracts 8 high-yield terminology pairs (Front: Concept/Prompt, Back: Definition/Formula).
- Implemented using pure CSS 3D transforms (`transform-style: preserve-3d`, `perspective: 1000px`, `backface-visibility: hidden`).
- Clicking the card triggers smooth 180-degree flip animations. Includes keyboard shortcuts and shuffle capabilities.

3.6 RAG Document Chat Tutor (`/api/chat`)

- Provides conversational question-answering strictly grounded in the uploaded document.
- Appends the last 6 conversation turns for natural dialogue context.
- **Zero Hallucination System Prompt:** The LLM is instructed: *"Answer strictly and only using the provided document text. If the answer cannot be found in the text, clearly state that the material does not cover it."*

3.7 Evaluator & Admin Oversight Portal (`/admin`)

- Accessible only by users with `role === "admin"`.
- Guarded by Next.js Middleware and `requireAdmin()` API checks.
- Provides aggregated KPI cards: Total Registered Students, Total System Documents, Active Quizzes.
- Full user management audit table: inspect student emails, verification states, join dates, and document upload counts.

---

4. Database Schema & Security Engineering

The application utilizes MongoDB Atlas with explicit compound and unique indexes to guarantee high query efficiency and atomic rate limiting.

4.1 Collections & Indexing Matrix

Collection	Key Document Fields	Indexes & Purpose
`users`	`name, email, password, role, isVerified, failedLoginAttempts, lockedUntil, createdAt`	`{ email: 1 }` (Unique index). Prevents duplicate registrations and enables sub-millisecond authentication lookups.
`documents`	`userId (ObjectId), filename, size, text, createdAt`	`{ userId: 1, createdAt: -1 }` (Compound index). Powers instant retrieval of a student's notes sorted chronologically.
`quizAttempts`	`userId, documentId, score, totalQuestions, answers, createdAt`	`{ userId: 1, createdAt: -1 }`. Drives score history charts and analytics.
`flashcardHistory`	`userId, documentId, cards, createdAt`	`{ userId: 1, createdAt: -1 }`. Saves generated flashcard decks for repeat drills.
`summaryHistory`	`userId, documentId, length, summary, createdAt`	`{ userId: 1, createdAt: -1 }`. Caches summaries so students don't re-incur AI latency.
`chatHistory`	`userId, documentId, messages, updatedAt`	`{ userId: 1, updatedAt: -1 }`. Maintains persistent conversational threads per document.
`rateLimits`	`key, count, expiresAt, createdAt`	`{ expiresAt: 1 }` (TTL index with `expireAfterSeconds: 0`). MongoDB automatically deletes expired rate limit windows!

4.2 Security Defenses Implemented

- **Insecure Direct Object Reference (IDOR) Prevention:** In `lib/auth-guard.ts`, the `getOwnedDocument(userId, documentId)` function validates that the requested `documentId` matches the authenticated session's `userId`. A malicious student cannot view or query another student's uploaded notes by tampering with document IDs.
- **Serverless Dev Connection Pooling:** In `lib/mongodb.ts`, the client promise is attached to `globalThis._mongoClientPromise` during development. This prevents Next.js Hot Module Reloading (HMR) from spawning hundreds of open database sockets.
- **Custom DNS SRV Fallback Resolver:** Node.js SRV lookups on Windows often fail when local DNS or ISP servers block SRV packets. StudyMate includes an automatic DNS fallback that queries Google Public DNS (`8.8.8.8`) and Cloudflare DNS (`1.1.1.1`) directly.
- **MongoDB-Backed Sliding Window Rate Limiter:** Without needing an expensive Redis cluster, `lib/rate-limit.ts` uses atomic `findOneAndUpdate` with `$inc` and MongoDB TTL indexes to limit AI generation requests to 15 per minute per student.

5. Master Viva Voce Questions & Model Answers

Study these questions thoroughly. They represent the exact technical questions external examiners and university professors ask during capstone project examinations.

Category A: Project Concept, Scope & Architecture

Q1 What is the core objective of StudyMate and what problem does it solve?

StudyMate is an AI-powered study companion designed for university students. It addresses the problem of cognitive overload and passive learning from massive course PDFs. It automates four evidence-based study methodologies: concise summarization, active-recall 3D flashcards, self-testing via multiple-choice quizzes with instant grading, and conversational query resolution using grounded RAG.

> **Viva Tip:** Mention the term "Pedagogical Efficacy" — examiners love hearing that your software is rooted in real educational theory (Active Recall & Spaced Repetition).

Q2 Explain the architectural flow when a student uploads a PDF.

1. The client sends a multipart FormData request to `/api/documents`.
2. Edge middleware checks if the user is authenticated; if so, passes to the route handler.
3. The route handler invokes `requireStudent()`, validating role and session validity.
4. The file buffer is extracted in memory and passed to `extractTextFromPdf()` in `lib/pdf.ts`.
5. `pdf-parse` extracts character streams, whitespace is normalized, and character length is validated.
6. Document metadata (name, size, timestamp) and sanitized text are stored in MongoDB under the `documents` collection linked to `userId`.

Q3 Why did you use Next.js 14 instead of a separate React frontend and Express.js backend?

Next.js 14 App Router provides a unified full-stack architecture with significant advantages:

- **Zero CORS complexity:** Frontend and API route handlers share the same domain and origin.
- **Server-Side Security:** Sensitive credentials (Groq API Key, MongoDB URI, SMTP passwords) reside purely in Node.js server execution contexts and are never leaked to client bundles.
- **Edge Middleware:** Allows instant route guarding and redirection before page rendering begins.
- **Developer Velocity & Type Safety:** Shared TypeScript interfaces across client components and server route handlers.

Q4 What design pattern does your frontend follow?

It follows the **Server/Client Component Boundary Pattern**. Static structural layouts, page skeletons, and metadata run as React Server Components (RSC) for maximum speed and SEO. Highly interactive forms (chat input, quiz state machines, flashcard flip cards) are isolated as `"use client"` components with local state hooks (`useState`, `useEffect`).

Category B: Artificial Intelligence, LLM & RAG Mechanics

Q5 How does Retrieval-Augmented Generation (RAG) work in this application?

RAG in StudyMate works by dynamically retrieving the student's authenticated, parsed PDF text from MongoDB and injecting it directly into the LLM system prompt as factual context. Rather than letting the LLM generate answers from general web training weights, the system instructions strictly constrain the model: "*Answer strictly and only using the provided document text.*" This grounds the output in course-specific syllabus notes and prevents hallucinations.

Q6 Why did you choose Groq Cloud with LLaMA 3.1 instead of OpenAI GPT-4?

Two main reasons: **Latency and Cost-Efficiency**.
1. **Sub-Second Latency:** Groq utilizes specialized LPUs (Language Processing Units) that achieve inference speeds of 300 to 500 tokens per second. Generating a 10-question quiz takes under 2 seconds on Groq, compared to 10-15 seconds on traditional GPU-hosted models.
2. **LLaMA 3.1 Performance:** Meta's open-weights LLaMA 3.1 model provides state-of-the-art reasoning, JSON schema compliance, and academic synthesis capabilities without vendor lock-in.

Q7 What happens if the student uploads a 200-page book? How do you prevent context overflow?

In `lib/groq.ts`, the function `shortenText()` enforces a strict budget (`MAX_TEXT_CHARS = 12,000` characters, roughly 3,000 tokens). If a document exceeds this limit, it is cleanly truncated with an explanatory marker `[Notes truncated for length]`. This ensures the prompt always fits comfortably within the model's context window while reserving ample token space for generated JSON responses.

Q8 How do you ensure the AI outputs strict JSON for quizzes and flashcards without breaking?

We employ a two-layer defense:
1. **API-Level JSON Schema Enforcement:** We pass `response_format: { type: "json_object" }` to the Groq API call, forcing the model's decoding logits to adhere to valid JSON syntax.
2. **Extraction & Sanitization Parser:** In `lib/groq.ts`, the `extractJson()` function strips any markdown code fences and extracts substrings between the first `{` and the last `}`, guaranteeing resilience against rogue conversational prefixes.

Q9 What is your fallback mechanism if a Groq model experiences downtime or rate limits?

In `lib/groq.ts`, we defined an array of fallback models: `["openai/gpt-oss-20b", "qwen/qwen3.8-27b", "openai/gpt-oss-120b"]`. If an HTTP 403, 404, 429, 500, 502, or 503 status code is returned, the loop automatically retries with the next alternative model, ensuring seamless uptime for the student.

Category C: Database, Storage & Backend Design

Q10 Why did you use the MongoDB Native Driver instead of Mongoose?

1. **Serverless Performance:** Mongoose establishes heavyweight schema caches and middleware hooks that cause memory spikes and cold-start latency in serverless environments.
2. **Lightweight & Raw Speed:** The official `mongodb` driver (v7.6.0) connects directly to MongoDB Atlas wire protocol with minimal overhead.
3. **Atomic Operations:** We can execute native atomic methods such as `findOneAndUpdate`, `$inc`, and `$setOnInsert` cleanly without Mongoose schema constraints.

Q11 How do you prevent multiple open database connections in development?

Next.js in development mode re-evaluates modules on every file save (Hot Module Reloading). To prevent socket exhaustion, in `lib/mongodb.ts` we cache the MongoClient promise on the global execution scope: `globalWithMongo._mongoClientPromise`. In production, a standard singleton promise is maintained.

Q12 Explain how the Rate Limiter works without Redis.

In `lib/rate-limit.ts`, we implemented an atomic MongoDB sliding-window rate limiter using a dedicated `rateLimits` collection. When a request arrives, it calls `findOneAndUpdate` with `$inc: { count: 1 }` and sets an expiration timestamp. A MongoDB **TTL (Time-To-Live) index** on `expiresAt` automatically instructs MongoDB's background thread to purge expired rate limit records with zero developer maintenance.

> **Viva Tip:** Examiners are often impressed when you demonstrate that you didn't need to pay for an external Redis cache to achieve production-grade rate limiting.

Q13 Why do you have a custom DNS resolution function in `lib/mongodb.ts`?

MongoDB Atlas connection strings use the `mongodb+srv://` protocol, which relies on DNS SRV records to discover cluster nodes. On certain networks (college Wi-Fi, firewalled VPNs, or Windows localhost), local DNS resolvers fail to resolve SRV records properly. Our `resolveMongoUri()` function uses Node.js `dns.promises.Resolver` with fallback upstream DNS servers (`8.8.8.8` and `1.1.1.1`) to resolve cluster hosts directly and construct a standard `mongodb://` connection string.

Category D: Authentication, Security & Cryptography

Q14 Why did you migrate from `bcrypt` to `bcryptjs`?

This was a critical production stability decision. Standard `bcrypt` is a native C++ module that compiles bindings during `npm install` using `node-gyp` and Python. When deploying across environments (e.g., developing on Windows x64 and deploying to Vercel's Linux container), binary ABI mismatches trigger the error: "*No native build was found for platform=linux arch=x64 runtime=node*". **`bcryptjs`** is written in 100% pure JavaScript, completely eliminating native compilation dependencies while maintaining identical cryptographic hashing security.

> **Viva Goldmine:** This is a real-world engineering challenge you solved. Highlighting this shows deep debugging competence.

Q15 How does password hashing work in StudyMate? What is a salt round?

Passwords are never stored in plaintext. During registration (`/api/register`), the password is processed using `bcryptjs.hash(password, 10)` .

The **cost factor (10 salt rounds)** means the algorithm computes 2^{10} (1,024) hashing iterations with a cryptographically secure random salt. This defends against rainbow table attacks and makes brute-force hardware cracking computationally prohibitive.

Q16 How is Role-Based Access Control (RBAC) enforced between Students and Admins?

RBAC is enforced across two complementary layers:

1. **Edge Middleware (`middleware.ts`):** Intercepts incoming HTTP requests. If a non-logged-in user hits `/dashboard` or `/admin` , they are redirected to `/login` . If a student tries to access `/admin` , they are bounced back to `/dashboard` .
2. **API Route Guards (`lib/auth-guard.ts`):** Every API route handler invokes `requireStudent()` or `requireAdmin()` . These inspect the session, verify the user's fresh database record, and return HTTP 401 or 403 JSON errors if unauthorized.

Q17 How does the OTP email verification flow work?

1. During registration, a 6-digit numeric token is generated via `crypto.randomInt(100000, 999999)` .
2. The token is hashed and saved in the `otps` collection with a 10-minute expiration.
3. Nodemailer connects via Gmail SMTP over TLS to dispatch the formatted verification code.
4. Upon entering the code in the UI, `/api/auth/otp/verify` checks the match, marks the user's `isVerified: true` flag in MongoDB, and cleans up the token.

Q18 What is an IDOR vulnerability, and how did you prevent it?

Insecure Direct Object Reference (IDOR) happens when an application uses student-supplied input (like a document ID in a JSON body) to access a database object without verifying ownership.

StudyMate prevents IDOR in `getOwnedDocument(userId, documentId)` by always including `userId: new ObjectId(userId)` in the database query. Even if Student A guesses Student B's document ID, MongoDB returns `null` , and the request is rejected with HTTP 404.

Category E: Frontend Engineering & UI/UX Innovations

Q19 How is the "Download Quiz Result (PDF)" feature implemented?

Instead of adding heavy client-side PDF generation packages like `jspdf` or `html2pdf` (which bloat the bundle by hundreds of kilobytes), StudyMate utilizes the browser's native print engine via `window.print()` paired with specialized **CSS Paged Media (`@media print`)** styling. When the user clicks "Download Result":

- Sidebars, navigation headers, and form buttons are hidden via `print:hidden`.
- A dedicated printable scorecard container (`print:block`) is revealed with student scores, percentage, date, and review cards.
- Cards apply `print-break-inside-avoid` so questions are never split across page boundaries.

Q20 How do the 3D Flashcards work under the hood?

They use modern CSS 3D Transforms:
- The outer card wrapper has `perspective: 1000px`.
- The inner container has `transform-style: preserve-3d` and a smooth transition.
- Front and back faces have `backface-visibility: hidden`.
- Clicking the card toggles a state variable that applies `transform: rotateY(180deg)`, seamlessly flipping from the concept to the definition without any external animation libraries.

Q21 What is the "Clean Luxe Academic" design system?

It is our custom Tailwind design system engineered to look like modern institutional software (Linear/Notion-inspired). It features:

- Tailored palette: Deep slate (`#0f172a`), Emerald brand accents (`#0f766e`, `#10b981`), and soft background canvas (`#f8fafc`).
- Subtle glassmorphism headers with `backdrop-blur-md`.
- Smooth micro-animations, clear hierarchy, and distinct accessible contrast ratios complying with WCAG 2.1 AA guidelines.

Category F: Limitations, Edge Cases & Future Enhancements

Q22 What are the current limitations of your system?

1. **Scanned Image PDFs:** `pdf-parse` extracts digital text streams. Scanned photocopies without an embedded OCR text layer will return empty text.
2. **File Size Limit:** Maximum upload size is currently capped at 5MB to preserve serverless memory limits.
3. **Mathematical Formatting:** Complex LaTeX formulas are parsed as raw strings rather than interactive MathJax/KaTeX equations.

> **Viva Tip:** Being honest about limitations demonstrates academic maturity. Never claim your system has zero flaws!

Q23 What future enhancements would you propose for Version 2.0?

- **Vector Embeddings & Semantic Search:** Integrate Pinecone or MongoDB Vector Search with LangChain or LlamaIndex to chunk multi-hundred-page textbooks and perform cosine-similarity retrieval.
- **OCR Engine:** Integrate Tesseract.js or AWS Textract to support handwritten lecture notes and image-only scans.
- **Audio Lecture Summaries:** Add Groq Whisper speech-to-text to transcribe recorded lecture audio files directly into study notes.
- **Collaborative Study Rooms:** Enable real-time shared quiz competitions and note sharing using WebSockets.

Q24 How did you test and validate the application?

We conducted multi-layer testing:

1. **Unit & Static Analysis:** TypeScript strict mode compiler checks (`tsc --noEmit`) and ESLint rules to eliminate type coercion and unused variables.
2. **Functional API Testing:** Tested all route handlers with positive and negative test cases (malformed payloads, unauthenticated sessions, oversized files, non-PDF types).
3. **Security Penetration Testing:** Tested IDOR attacks by attempting to query another student's document ID, verified account lockout after 6 invalid passwords, and verified rate limiter triggering.
4. **Cross-Browser & Responsive QA:** Verified desktop and mobile viewport rendering across Chrome, Edge, Firefox, and iOS Safari.

6. Viva Day Emergency Cheat Sheet

> **Keep these numbers and facts memorized for instant recall when questioned by examiners:**

Metric / Concept	Exact Specification	Where in Code
Maximum PDF Size	5 MB (5 * 1024 * 1024 bytes)	`lib/constants.ts`
Max Text Passed to AI	12,000 characters (~3,000 tokens)	`lib/constants.ts`
Default Quiz Questions	5 questions (configurable up to 30)	`app/api/quiz/route.ts`
Default Flashcard Deck	8 high-yield cards	`lib/constants.ts`
Password Salt Rounds	10 cost rounds (bcryptjs)	`lib/auth.ts`, `lib/ensure-admin.ts`
Brute-Force Lockout	6 failed attempts = 60-second lock	`lib/auth.ts`
OTP Expiration	10 minutes (600 seconds)	`app/api/auth/otp/send/route.ts`
Rate Limit Window	15 requests per 60 seconds (Sliding Window)	`lib/rate-limit.ts`
Default Admin Email	admin@studymate.local	`lib/ensure-admin.ts`
AI Engine & Protocol	Groq Cloud REST API (LLaMA 3.1)	`lib/groq.ts`

Examiner "Gotcha" Questions & Quick Defenses

- **Examiner:** "*What if the student submits empty credentials?*"
  Answer: `lib/auth.ts` immediately throws "*Missing email or password*" and validates against an RFC 5322 email regex before touching the database.
- **Examiner:** "*Is your database vulnerable to SQL injection?*"
  Answer: "*We use MongoDB, which uses binary JSON (BSON) query objects rather than text SQL concatenation. Furthermore, all query identifiers are strictly cast to* `new ObjectId()` *with validation via* `ObjectId.isValid()`*, completely preventing NoSQL injection.*"
- **Examiner:** "*Why didn't you use local Ollama instead of Groq?*"
  Answer: "*Local Ollama requires high-end server GPUs (16GB+ VRAM) which makes cloud hosting expensive and unavailable on serverless platforms like Vercel. Groq provides fast API inference on specialized LPU hardware with zero local GPU infrastructure requirements.*"
